

Gemma4 模型架构

目 录

1 引言：Gemma4 的改进方向	2
2 模型参数总览	2
3 滑动窗口注意力	2
3.1 动机	2
3.2 直觉	2
3.3 形式化	3
3.4 回看	3
4 分组查询注意力（GQA）	3
4.1 动机	3
4.2 Gemma4 中的 GQA 配置	3
5 SwiGLU 前馈网络	4
5.1 动机	4
5.2 SiLU 激活函数	4
5.3 SwiGLU	4
6 混合注意力架构	4
6.1 动机	4
6.2 全局层的特殊设计	4
6.3 代码接口	5
7 p-RoPE 位置编码	5
7.1 动机	5
7.2 原理	5
8 实战：注意力矩阵大小对比	5
9 要点速查	6
10 本章你将学会	6
11 小结	6

1 引言：Gemma4 的改进方向

标准 Transformer 的注意力让每个 token 看到序列中所有其他 token，效果虽好但开销随序列长度平方增长。当模型参数量到 12B 级别、词表超过 26 万时，如果直接套用标准架构，推理时的计算和显存开销都会爆炸。

Gemma4-12B 在标准 Decoder-only 架构上引入了四项关键改进：**滑动窗口注意力**把大部分层的注意力范围限制在局部窗口，**分组查询注意力**（GQA）让多个 Query 头共享 KV 头以压缩 KV Cache，**SwiGLU** 前馈网络用门控机制提升表达能力，**混合注意力**在滑动窗口层中穿插少量全局层来捕捉长程依赖。这些改进共同作用，使 Gemma4-12B 在保持精度的同时大幅降低了推理开销。

本章我们逐一拆解这些设计，理解它们如何影响 KV Cache 大小和 GPU 利用率，为后续的量化和推理优化打下基础。

2 模型参数总览

Gemma4-12B 的主要参数如下表所示。

参数	值	含义	简写
num_hidden_layers	48	Decoder Layer 数量	N
hidden_size	3840	隐藏层维度	D
num_attention_heads	16	注意力头数量	H_q
num_key_value_heads	8	滑动窗口层 KV 头数	$H_{l,kv}$
num_global_key_value_heads	1	全局注意力层 KV 头数	$H_{g,kv}$
head_dim	256	滑动窗口层每头维度	$D_{l,h}$
global_head_dim	512	全局注意力层每头维度	$D_{g,h}$
sliding_window	1024	滑动窗口大小	w
intermediate_size	15360	FFN 中间层维度	F
vocab_size	262144	词汇表大小	V

注意 Gemma4 中 Query 头维度 $H_q \times D_h = 16 \times 256 = 4096$ ，大于隐藏维度 $D = 3840$ 。这意味着 Q 投影是一个“升维”投影，从 3840 维映射到 4096 维。

3 滑动窗口注意力

3.1 动机

标准注意力的每个 token 要关注序列中所有 token，计算量和显存都是 $O(S^2)$ 。当序列长度 S 达到数千时，这个开销非常大。实际应用中，语言模型的大部分注意力集中在局部上下文，远处的 token 影响很小。滑动窗口注意力正是利用这一局部性。

3.2 直觉

想象你在读一本书，读到第 1000 页时，最近几页的内容对你理解当前段落最重要，第 1 页的细节反而影响不大。滑动窗口注意力让每个 token 只“看”它之前 w 个 token，把注意力范围从全局缩小到局部窗口。

3.3 形式化

标准自注意力的计算为：

$$O = \text{softmax}\left(\frac{A + M}{\sqrt{d_k}}\right)V \quad (3.1)$$

其中 $A = QK^T$ 是注意力分数， M 是掩码矩阵。在因果注意力中， M 把未来位置的分数设为 $-\infty$ 。滑动窗口注意力把 M 改为条带状：位置 i 只关注 $j \in [\max(0, i - w + 1), i]$ ，窗口外的位置分数也设为 $-\infty$ 。

这样，每个 token 实际参与计算的注意力条目从 S 个减少到 w 个，注意力计算量从 $O(S^2)$ 降为 $O(Sw)$ 。

直觉 标准注意力像“全景照片”，每个 token 看到全部上下文；滑动窗口像“放大镜”，每个 token 只看最近 w 个 token。视野窄了，但速度快了，而且大部分信息本来就在局部窗口内。

3.4 回看

滑动窗口以 $w = 1024$ 的窗口限制注意力范围。对于 $S = 2048$ 的序列，每个 token 最多看 1024 个前驱 token，计算量约为标准注意力的一半。但仅有局部注意力无法捕捉长程依赖，这需要混合注意力中的全局层来补充。

4 分组查询注意力（GQA）

4.1 动机

MQA（Multi-Query Attention）让所有 Query 头共享 1 个 KV 头，KV Cache 最小但精度损失大。**MHA**（Multi-Head Attention）每个 Query 头配 1 个 KV 头，精度最好但 KV Cache 最大。**GQA**（Grouped-Query Attention）是折中方案：多个 Query 头共享 1 个 KV 头，在精度和效率之间取得平衡。

4.2 Gemma4 中的 GQA 配置

Gemma4-12B 对两类注意力层使用不同的 GQA 分组：

层类型	Query 头	KV 头	共享比例
滑动窗口层	$H_q = 16$	$H_{l,kv} = 8$	2:1
全局层	$H_q = 16$	$H_{g,kv} = 1$	16:1

滑动窗口层每 2 个 Query 头共享 1 个 KV 头，全局层所有 16 个 Query 头共享 1 个 KV 头。全局层的 KV Cache 更小，因为只有 1 个 KV 头。

在实际计算时，KV 头的 Key 和 Value 需要通过 `repeat_kv` 操作复制到与 Query 头数相同的数量，才能做注意力计算。这个复制不增加 KV Cache 的存储，只增加计算时的临时显存。

5 SwiGLU 前馈网络

5.1 动机

标准 FFN 用两个线性层加激活函数，表达力有限。GLU (Gated Linear Units, 门控线性单元) 引入门控机制：一部分网络计算“值”，另一部分计算“门”，两者相乘决定输出。

SwiGLU 用 SiLU 作为门控激活，在多个大规模模型中表现优异。

5.2 SiLU 激活函数

SiLU (Sigmoid Linear Unit) 定义为：

$$\text{SiLU}(x) = x \cdot \sigma(x) \quad (5.1)$$

其中 $\sigma(x) = \frac{1}{1+e^{-x}}$ 是 sigmoid 函数。SiLU 在 x 较大时接近 x (类似 ReLU)，在 x 较小时允许少量负值通过，梯度更平滑。

5.3 SwiGLU

SwiGLU 的计算为：

$$\text{SwiGLU}(x) = \text{SiLU}(W_g x) \odot (W_u x) \quad (5.2)$$

FFN 包含三个线性投影：

投影	权重形状	作用
gate	$D \times F$	计算门控信号
up	$D \times F$	计算值信号
down	$F \times D$	降维回 D

其中 $D = 3840$, $F = 15360$ 。gate 和 up 把 D 维升到 F 维，SiLU 作用在 gate 上，两者逐元素相乘后经 down 降回 D 维。三个投影的权重总量为 $3DF$ ，是 FFN 显存占用的主要来源。

直觉 SwiGLU 像一个“智能开关”：gate 路径决定哪些信息通过，up 路径准备要传递的内容，两者相乘实现选择性传递。这比标准 FFN 的“全开”或“全关”更灵活。

6 混合注意力架构

6.1 动机

纯滑动窗口注意力无法捕捉超出窗口 w 的长程依赖。如果所有层都用全局注意力，开销又太大。混合注意力把大部分层设为滑动窗口，每隔几层穿插一个全局层，兼顾局部效率和长程能力。

6.2 全局层的特殊设计

Gemma4 的全局层有一个特别之处：Key 和 Value 共享同一组权重，即 `attention_k_eq_v = True`。这意味着 W_K 和 W_V 是同一个矩阵，进一步减少了全局层的参数量。

全局层还使用 **p-RoPE** (Proportional Rotary Position Embedding, 比例旋转位置编码), 为全局注意力提供位置信息。p-RoPE 对不同注意力头使用不同频率的旋转, 低频头捕捉长程位置关系, 高频头捕捉短程位置关系。

6.3 代码接口

在框架中, DecoderLayer 根据 layer_types[layer_idx] 选择使用哪种注意力层:

```
class DecoderLayer(nn.Module):
    def __init__(self, config, layer_idx):
        super().__init__()
        layer_type = config.layer_types[layer_idx]
        if layer_type == "sliding_attention":
            self.self_attn = SlidingAttentionLayer(config, layer_idx)
        else:
            self.self_attn = AttentionLayer(config, layer_idx)
        self.mlp = SwiGLU(config)
```

layer_types 是一个长度为 N 的列表, 指定每一层的类型。SlidingAttentionLayer 实现滑动窗口掩码, AttentionLayer 实现全局注意力。

7 p-RoPE 位置编码

7.1 动机

标准 RoPE 对所有头使用相同频率, 低频头捕捉短程关系, 高频头捕捉长程关系。但实验发现, 某些头需要更低的频率来感知极远距离的位置。p-RoPE 通过比例缩放, 让部分头使用更低的旋转频率, 从而扩展位置感知范围。

7.2 原理

RoPE 对 Query 和 Key 的每对维度施加旋转矩阵, 将位置信息编码到向量方向中。旋转角度为 $\theta_{i,j} = \frac{1}{10000^{2\frac{j}{d_h}}}$, 其中 i 是头的编号, j 是维度对的编号。频率随 j 增大而升高, 低频头感知远距离位置, 高频头感知近距离位置。

p-RoPE 对全局层的某些头使用缩放后的 θ , 降低频率以感知更远的位置。

p-RoPE 的具体实现细节不在本实验范围内, 你只需要知道全局层使用 *p-RoPE* 提供位置信息, 滑动窗口层因窗口有限, 对位置编码的远端精度要求较低。

8 实战: 注意力矩阵大小对比

我们比较滑动窗口注意力和全局注意力在序列长度 $S = 2048$ 时的注意力矩阵规模。设 $w = 1024$ 。

例 全局注意力: 每个 token 关注所有前驱 token, 注意力矩阵大小为 $S \times S$:

$$2048 \times 2048 = 4,194,304 \text{ 个元素} \quad (8.1)$$

滑动窗口注意力: 每个 token 最多关注 w 个前驱, 有效元素约为 $S \times w$:

$$2048 \times 1024 = 2,097,152 \text{ 个元素} \quad (8.2)$$

比值: $2,097, \frac{152}{4}, 194,304 = 50\%$, 滑动窗口的计算量约为全局注意力的一半。

KV Cache 对比 (单层, BF16):

全局层 KV Cache: $2 \times 1 \times 2048 \times 1 \times 512 \times \frac{16}{8} = 524,288 \text{ bytes} = 512 \text{ KiB}$

滑动窗口层 KV Cache: $2 \times 1 \times 2048 \times 8 \times 256 \times \frac{16}{8} = 2,097,152 \text{ bytes} = 2 \text{ MiB}$

可以看到, 全局层虽然每个头的维度更大 (512 vs 256), 但 KV 头数只有 1 (vs 8), 所以 KV Cache 反而更小。

9 要点速查

特性	设计	效果
滑动窗口	$w = 1024$ 局部注意力	计算量 $O(Sw)$
GQA (滑动)	$H_q : H_{l,kv} = 2 : 1$	KV Cache 减半
GQA (全局)	$H_q : H_{g,kv} = 16 : 1$	KV Cache 极小
SwiGLU	SiLU 门控 FFN	3DF 参数
全局层 $W_K = W_V$	K/V 共享权重	参数减半
混合注意力	大部分滑动 + 少量全局	兼顾效率与长程

10 本章你将学会

1. 列出 Gemma4-12B 的主要参数, 解释 H_q 、 $H_{l,kv}$ 、 $H_{g,kv}$ 的含义和比例关系。
2. 描述滑动窗口注意力的掩码设计, 计算给定 S 和 w 下的注意力计算量。
3. 解释 GQA 如何减少 KV Cache 大小, 说明 repeat_kv 的作用。
4. 写出 SwiGLU 的公式和三个投影的形状, 计算 FFN 的参数量。
5. 说明混合注意力架构中全局层和滑动窗口层的分工, 以及全局层 $W_K = W_V$ 的设计。

11 小结

Gemma4-12B 通过四项关键改进优化推理效率: 滑动窗口注意力把大部分层的注意力计算从 $O(S^2)$ 降为 $O(Sw)$; GQA 让滑动窗口层 2:1、全局层 16:1 共享 KV 头, 压缩 KV Cache; SwiGLU 用门控机制提升 FFN 表达力; 混合注意力在局部层中穿插全局层, 兼顾效率和长程依赖。这些设计共同决定了模型的显存占用和计算瓶颈, 下一章我们将学习如何通过量化进一步压缩权重显存。

讲义基于 HPC Lab5 实验指导编写